

Phase 1 Notes: Binary Tree Foundations, Nodes, Recursion, Count, and Height

1. Main Goal of Phase 1

Phase 1 teaches the basic foundation needed before learning Binary Search Trees.

Before we can search, insert, or delete in a Binary Search Tree, we must first understand how a normal binary tree is stored in memory and how recursion works on tree-shaped data.

By the end of this phase, you should understand:

- What a binary tree is.
 - What a node is.
 - What `lchild`, `data`, and `rchild` mean.
 - What `NULL` means in a tree.
 - What root, child, subtree, and leaf mean.
 - Why recursion is useful for trees.
 - How to count all nodes in a tree.
 - How to find the height of a tree.
 - Why count and height take $O(n)$ time.
 - Why recursive tree functions use $O(h)$ stack space.
-

2. What Is a Binary Tree?

A binary tree is a structure made of nodes.

Each node can have at most two children:

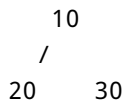
1. A left child.
2. A right child.

The word **binary** means two. So a binary tree is a tree where each node can have a maximum of two child nodes.

A node may have:

- No children.
- Only a left child.
- Only a right child.
- Both a left and a right child.

Example:



Here:

- 10 is a node.
- 20 is the left child of 10.
- 30 is the right child of 10.

3. Binary Tree Node Structure in C

In C, a binary tree node can be written like this:

```
struct Node {  
    struct Node *lchild;  
    int data;  
    struct Node *rchild;  
};
```

This structure has three parts.

3.1 lchild

```
struct Node *lchild;
```

This stores the address of the left child node.

If the node has no left child, lchild should be NULL.

3.2 data

```
int data;
```

This stores the actual value inside the node.

Example:

```
10, 20, 30, 40
```

These values are stored in the `data` field of different nodes.

3.3 `rchild`

```
struct Node *rchild;
```

This stores the address of the right child node.

If the node has no right child, `rchild` should be `NULL`.

4. Meaning of `NULL` in a Tree

`NULL` means there is no node at that place.

For example:

```
    10
   /
  20
```

Node `10` has a left child, but no right child.

So in memory:

```
root->lchild points to node 20
root->rchild is NULL
```

A `NULL` child pointer means the tree does not continue in that direction.

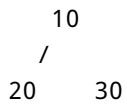
This is very important because recursive tree functions usually stop when they reach `NULL`.

5. Important Tree Terms

5.1 Root

The root is the top node of the tree.

Example:



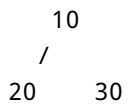
Here, **10** is the root.

The root is where we usually start when working with a tree.

5.2 Child

A child is a node directly connected below another node.

Example:



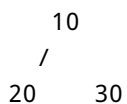
Here:

- **20** is the left child of **10**.
 - **30** is the right child of **10**.
-

5.3 Parent

A parent is the node directly above another node.

In this tree:

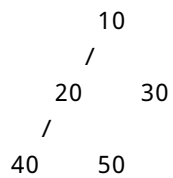


10 is the parent of **20** and **30**.

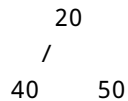
5.4 Subtree

A subtree is a smaller tree inside a bigger tree.

Example:



The subtree starting at 20 is:



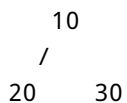
So a subtree is simply a node and everything below it.

5.5 Leaf Node

A leaf node is a node with no children.

That means both its left and right child pointers are NULL.

Example:



Here, 20 and 30 are leaf nodes because they have no children.

In code, a leaf node satisfies:

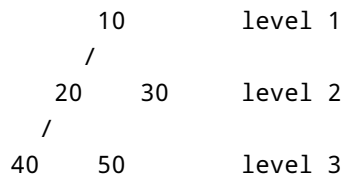
```
node->lchild == NULL && node->rchild == NULL
```

5.6 Level

A level tells how far a node is from the root.

In this curriculum, we can think of the root as level 1.

Example:



5.7 Height

Height is the number of levels in the longest path from a node down to a leaf.

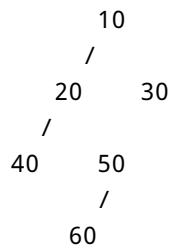
In this phase, height is counted by levels.

So:

- An empty tree has height .
- A single node has height .
- A tree with root and one child has height .

6. Example Tree Used in Phase 1

We will use this tree:



This tree has these nodes:

10, 20, 30, 40, 50, 60

So the total number of nodes is:

6

The longest path is:

10 -> 20 -> 50 -> 60

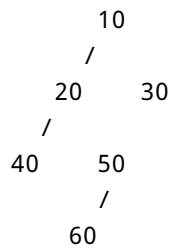
This path has 4 levels.

So the height is:

4

7. How the Example Tree Is Connected

The tree is:



The connections are:

10 has left child 20
10 has right child 30
20 has left child 40
20 has right child 50
50 has left child 60

The leaf nodes are:

40, 60, 30

Why?

Because:

- 40 has no left or right child.
- 60 has no left or right child.
- 30 has no left or right child.

8. Creating a Node in C

A helper function can be used to create a new node.

```
struct Node* createNode(int value) {
    struct Node *p = (struct Node*)malloc(sizeof(struct Node));

    if (p == NULL) {
        printf("Memory allocation failed\n");
        exit(1);
    }

    p->data = value;
    p->lchild = NULL;
    p->rchild = NULL;

    return p;
}
```

Explanation

```
struct Node *p = (struct Node*)malloc(sizeof(struct Node));
```

This creates memory for one new node.

```
p->data = value;
```

This stores the value inside the node.

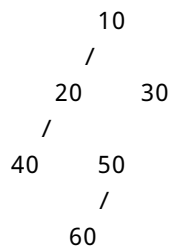
```
p->lchild = NULL;
p->rchild = NULL;
```

This means the new node does not have any children yet.

This is important. If we do not initialize child pointers to `NULL`, they may contain garbage addresses.

9. Manually Building the Example Tree

To build this tree:



We can write:

```
struct Node *root = createNode(10);

root->lchild = createNode(20);
root->rchild = createNode(30);

root->lchild->lchild = createNode(40);
root->lchild->rchild = createNode(50);

root->lchild->rchild->lchild = createNode(60);
```

Explanation

```
struct Node *root = createNode(10);
```

Creates the root node with value **10**.

```
root->lchild = createNode(20);
```

Creates node **20** and connects it as the left child of **10**.

```
root->rchild = createNode(30);
```

Creates node **30** and connects it as the right child of **10**.

```
root->lchild->lchild = createNode(40);
```

Goes to node **20**, then connects node **40** as its left child.

```
root->lchild->rchild = createNode(50);
```

Goes to node 20, then connects node 50 as its right child.

```
root->lchild->rchild->lchild = createNode(60);
```

Goes from 10 to 20, then to 50, then connects node 60 as the left child of 50.

10. Why Recursion Is Useful for Trees

Recursion means a function calls itself.

Trees work very well with recursion because every child of a node is also the root of a smaller tree.

For example:

```
    10
   /
  20  30
```

From the point of view of node 10:

- The left side is a smaller tree starting at 20.
- The right side is a smaller tree starting at 30.

So we can solve tree problems like this:

```
Solve the left subtree.
Solve the right subtree.
Combine the answers with the current node.
```

This is the basic idea behind many tree algorithms.

11. The Most Important Recursion Rule: Base Case

Every recursive function needs a stopping condition.

In tree recursion, the most common stopping condition is:

```
if (root == NULL)
```

This means:

There is no node here, so stop.

For counting nodes, if the pointer is `NULL`, the answer is `0`.

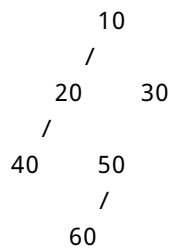
For height, if the pointer is `NULL`, the answer is also `0`.

Without this base case, the function may try to access a non-existing node and crash.

12. Counting Nodes in a Binary Tree

Counting nodes means finding how many real nodes exist in the tree.

For the example tree:



The nodes are:

10, 20, 30, 40, 50, 60

So the count is:

6

13. Logic for Counting Nodes

At each node, ask three questions:

1. How many nodes are in the left subtree?
2. How many nodes are in the right subtree?
3. How do I count the current node?

The answer is:

```
left count + right count + 1
```

The `+ 1` counts the current node.

Formula:

```
count(root) = count(root->lchild) + count(root->rchild) + 1
```

But if `root` is `NULL`:

```
count(NULL) = 0
```

14. Code for Counting Nodes

```
int count(struct Node *root) {  
    if (root != NULL) {  
        return count(root->lchild) + count(root->rchild) + 1;  
    }  
    return 0;  
}
```

Simple Explanation

```
if (root != NULL)
```

Means:

If there is a real node here, count it.

```
count(root->lchild)
```

Counts all nodes on the left side.

```
count(root->rchild)
```

Counts all nodes on the right side.

```
+ 1
```

Counts the current node.

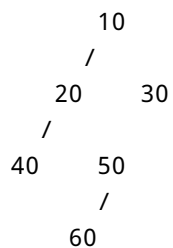
```
return 0;
```

Means:

If root is NULL, there is no node, so return 0.

15. Dry Run of `count()`

Tree:



Start from the bottom nodes.

Node 40

`40` has no left child and no right child.

```
count(40) = count(NULL) + count(NULL) + 1
count(40) = 0 + 0 + 1
count(40) = 1
```

Node 60

60 also has no children.

```
count(60) = 0 + 0 + 1
count(60) = 1
```

Node 50

50 has left child 60 and no right child.

```
count(50) = count(60) + count(NULL) + 1
count(50) = 1 + 0 + 1
count(50) = 2
```

Node 20

20 has left child 40 and right child 50.

```
count(20) = count(40) + count(50) + 1
count(20) = 1 + 2 + 1
count(20) = 4
```

Node 30

30 has no children.

```
count(30) = 0 + 0 + 1
count(30) = 1
```

Node 10

10 has left subtree rooted at 20 and right subtree rooted at 30.

```
count(10) = count(20) + count(30) + 1
count(10) = 4 + 1 + 1
count(10) = 6
```

Final answer:

Total nodes = 6

16. Finding the Height of a Binary Tree

Height means the number of levels in the longest path from the current node down to a leaf.

For the example tree:

10	level 1
/	
20 30	level 2
/	
40 50	level 3
/	
60	level 4

The longest path is:

10 -> 20 -> 50 -> 60

This path has 4 levels.

So the height is:

4

17. Important Height Rules

In this curriculum:

```
height(NULL) = 0
```

An empty tree has height .

```
height(single node) = 1
```

A single real node has height .

Example:

```
10
```

The height is .

Example:

```
10
 /
20
```

The height is .

18. Logic for Finding Height

To find height, ask:

1. What is the height of the left subtree?
2. What is the height of the right subtree?
3. Which side is taller?
4. Add 1 for the current node.

Formula:

```
height(root) = max(height(left), height(right)) + 1
```

We use because height cares about the longest path only.

We do not add both sides together.

19. Code for Height

```
int height(struct Node *root) {
    int x = 0, y = 0;

    if (root == NULL) {
        return 0;
    }

    x = height(root->lchild);
    y = height(root->rchild);

    if (x > y) {
        return x + 1;
    } else {
        return y + 1;
    }
}
```

Simple Explanation

```
if (root == NULL) {
    return 0;
}
```

If there is no node, the height is 0.

```
x = height(root->lchild);
```

Find the height of the left side.

```
y = height(root->rchild);
```

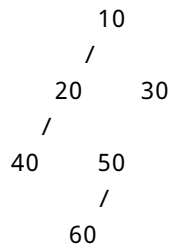
Find the height of the right side.

```
if (x > y) {
    return x + 1;
} else {
    return y + 1;
}
```

Return the taller side plus 1 for the current node.

20. Dry Run of `height()`

Tree:



Node 40

40 has no children.

```
height(40) = max(0, 0) + 1  
height(40) = 1
```

Node 60

60 has no children.

```
height(60) = max(0, 0) + 1  
height(60) = 1
```

Node 50

50 has left child 60 and no right child.

```
height(50) = max(height(60), height(NULL)) + 1  
height(50) = max(1, 0) + 1  
height(50) = 2
```

Node 20

20 has left child 40 and right child 50.

```
height(20) = max(height(40), height(50)) + 1
height(20) = max(1, 2) + 1
height(20) = 3
```

Node 30

30 has no children.

```
height(30) = max(0, 0) + 1
height(30) = 1
```

Node 10

10 has left subtree rooted at 20 and right subtree rooted at 30.

```
height(10) = max(height(20), height(30)) + 1
height(10) = max(3, 1) + 1
height(10) = 4
```

Final answer:

```
Height = 4
```

21. Difference Between Count and Height

Count and height both use recursion, but they are not the same.

Count

Count wants the total number of nodes.

So count uses both sides:

```
count = left count + right count + 1
```

Height

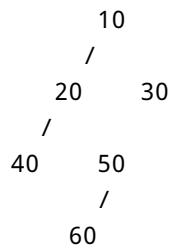
Height wants the longest path.

So height chooses only the taller side:

```
height = max(left height, right height) + 1
```

22. Count vs Height Example

Tree:



Count uses all nodes

```
10, 20, 30, 40, 50, 60
```

So:

```
count = 6
```

Height uses only the deepest path

```
10 -> 20 -> 50 -> 60
```

So:

```
height = 4
```

23. Complete C Program for Phase 1

```
#include <stdio.h>
#include <stdlib.h>
```

```

struct Node {
    struct Node *lchild;
    int data;
    struct Node *rchild;
};

struct Node* createNode(int value) {
    struct Node *p = (struct Node*)malloc(sizeof(struct Node));

    if (p == NULL) {
        printf("Memory allocation failed\n");
        exit(1);
    }

    p->data = value;
    p->lchild = NULL;
    p->rchild = NULL;

    return p;
}

int count(struct Node *root) {
    if (root != NULL) {
        return count(root->lchild) + count(root->rchild) + 1;
    }
    return 0;
}

int height(struct Node *root) {
    int x = 0, y = 0;

    if (root == NULL) {
        return 0;
    }

    x = height(root->lchild);
    y = height(root->rchild);

    return (x > y) ? x + 1 : y + 1;
}

int main() {
    struct Node *root = createNode(10);

    root->lchild = createNode(20);
    root->rchild = createNode(30);
}

```

```

    root->lchild->lchild = createNode(40);
    root->lchild->rchild = createNode(50);

    root->lchild->rchild->lchild = createNode(60);

    printf("Total nodes = %d\n", count(root));
    printf("Height = %d\n", height(root));

    return 0;
}

```

24. Program Output

```

Total nodes = 6
Height = 4

```

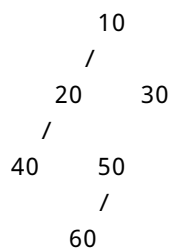
This output means:

- The tree contains 6 real nodes.
- The longest path has 4 levels.

25. Time Complexity of `count()`

The `count()` function must visit every node once.

For example, in this tree:



It must visit:

```

10, 20, 30, 40, 50, 60

```

So if the tree has `n` nodes, `count()` does work for all `n` nodes.

Therefore:

Time complexity of `count()` = $O(n)$

26. Time Complexity of `height()`

The `height()` function also visits every node.

Even though `height` only returns the longest path, it still has to check both the left and right sides to know which side is taller.

So if the tree has `n` nodes:

Time complexity of `height()` = $O(n)$

27. Recursion Stack Space

Recursive functions use the call stack.

Each time a function calls itself, one new function call is placed on the stack.

For tree recursion, the maximum number of active calls depends on the height of the tree.

So the stack space is:

$O(h)$

where `h` is the height of the tree.

28. Why Stack Space Is $O(h)$

The recursion usually goes down one path before returning.

For the example tree, the longest path is:

```
10 -> 20 -> 50 -> 60
```

This path has height `4`.

At the deepest point, the call stack may contain calls for:

```
height(10)
height(20)
height(50)
height(60)
```

That is why the stack space depends on the height.

So:

```
Stack space =  $O(h)$ 
```

29. Balanced Tree vs Skewed Tree Stack Space

If a tree is balanced, its height is small.

For a balanced tree:

```
h is about  $\log n$ 
```

So recursive stack space is about:

```
 $O(\log n)$ 
```

If a tree is skewed, it looks like a linked list.

Example:

```
10
  20
    30
```


40

Here, the height can become `n`.

So recursive stack space can become:

$O(n)$

In general, we say:

Recursive stack space = $O(h)$

30. Common Beginner Mistakes

Mistake 1: Forgetting the `NULL` Base Case

Wrong:

```
int count(struct Node *root) {  
    return count(root->lchild) + count(root->rchild) + 1;  
}
```

Problem:

If `root` becomes `NULL`, the code tries to access:

`root->lchild`

That can crash the program.

Correct:

```
int count(struct Node *root) {  
    if (root == NULL) {  
        return 0;  
    }  
}
```

```
    return count(root->lchild) + count(root->rchild) + 1;
}
```

Mistake 2: Confusing Count and Height

Wrong idea:

```
height = left height + right height + 1
```

This is not height.

That adds both sides, which is similar to counting.

Correct idea:

```
count = left count + right count + 1
height = max(left height, right height) + 1
```

Mistake 3: Thinking a Leaf Has Height 0

Some books count height by edges. In that definition, a leaf may have height 0.

But in this phase, height is counted by levels.

So:

```
height(NULL) = 0
height(leaf node) = 1
```

Always check which definition is being used.

Mistake 4: Not Initializing Child Pointers

Wrong:

```
p->data = value;
```

Problem:

The child pointers may contain garbage addresses.

Correct:

```
p->data = value;  
p->lchild = NULL;  
p->rchild = NULL;
```

Always set new node child pointers to `NULL`.

Mistake 5: Assuming Recursive Space Is Always $O(n)$

Recursive stack space is not always $O(n)$.

It depends on the height of the tree.

Correct:

```
Recursive stack space =  $O(h)$ 
```

If the tree is balanced:

```
 $O(h) = O(\log n)$ 
```

If the tree is skewed:

```
 $O(h) = O(n)$ 
```

31. Key Formulas to Remember

Count Formula

```
count(root) = count(root->lchild) + count(root->rchild) + 1
```

Base case:

```
count(NULL) = 0
```

Height Formula

```
height(root) = max(height(root->lchild), height(root->rchild)) + 1
```

Base case:

```
height(NULL) = 0
```

32. Mini Practice Example

Given this tree:

```
    5
   /
  3  8
```

Count

Nodes are:

```
5, 3, 8
```

So:

```
count = 3
```

Height

Levels:

```
    5      level 1
   /
  3  8    level 2
```

So:

```
height = 2
```

33. Another Practice Example

Given this tree:

```
  1
   \
    2
     \
      3
```

Count

Nodes are:

```
1, 2, 3
```

So:

```
count = 3
```

Height

The longest path is:

```
1 -> 2 -> 3
```

So:

```
height = 3
```

This tree is skewed because it grows in one direction.

34. Phase 1 Final Summary

A binary tree is made of nodes.

Each node has:

```
left child pointer  
value/data  
right child pointer
```

In C:

```
struct Node {  
    struct Node *lchild;  
    int data;  
    struct Node *rchild;  
};
```

If a child does not exist, its pointer is `NULL`.

The root is the top node.

A leaf is a node with no children.

A subtree is a smaller tree inside a bigger tree.

Recursion works well with trees because each child is the root of a smaller tree.

To count nodes:

```
count = left count + right count + 1
```

To find height:

```
height = max(left height, right height) + 1
```

Both `count()` and `height()` visit every node.

So their time complexity is:

$O(n)$

Their recursive stack space is:

$O(h)$

where h is the height of the tree.

35. What You Should Be Able to Do After Phase 1

After studying Phase 1, you should be able to:

- Draw a simple binary tree.
 - Identify the root of a tree.
 - Identify parent and child nodes.
 - Identify leaf nodes.
 - Explain what `NULL` means in a tree.
 - Write the C structure for a binary tree node.
 - Create a node using `malloc`.
 - Initialize child pointers to `NULL`.
 - Build a small tree manually using pointers.
 - Explain why recursion is natural for trees.
 - Write a recursive function to count nodes.
 - Write a recursive function to find height.
 - Dry run `count()` on a small tree.
 - Dry run `height()` on a small tree.
 - Explain why count and height are $O(n)$.
 - Explain why recursion stack space is $O(h)$.
-

36. Final Mental Model

Think of every node as asking two smaller versions of the same question.

For count:

Left child, how many nodes do you have?
Right child, how many nodes do you have?
I will add 1 for myself.

For height:

Left child, how tall are you?
Right child, how tall are you?
I will choose the taller one and add 1 for myself.

That is the heart of Phase 1.